

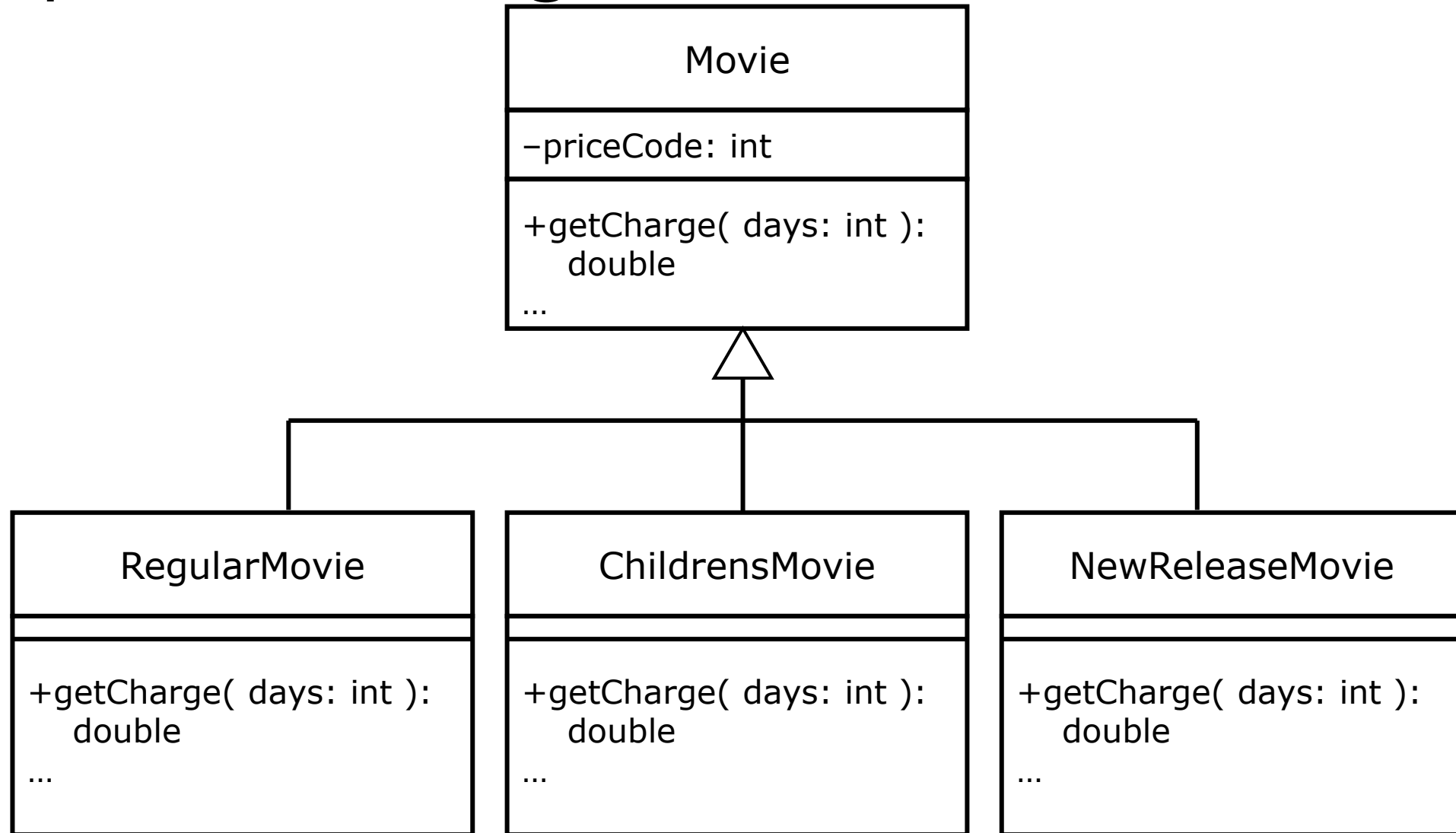
Replace Conditional Logic

- Ready for inheritance? ...

Take home exercise.

Use your design patterns and inheritance logic to apply inheritance.

Proposed Redesign?



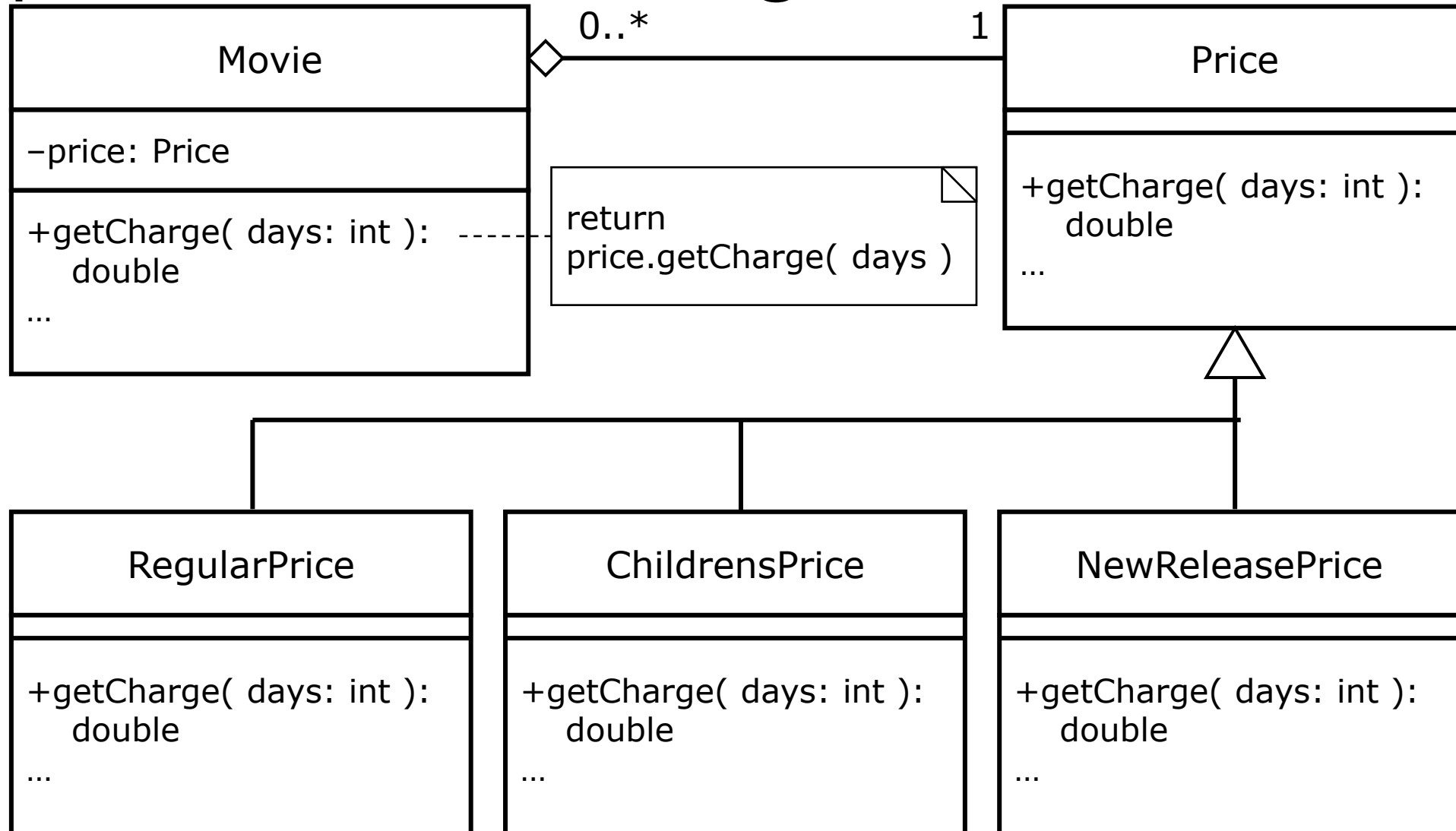
Proposed Redesign

- Flaw:
 - A movie may change its classification during its lifetime (e.g., new release to regular)
 - But an object cannot change its class during its lifetime
 - Solution?

Replace Conditional Logic

- Idea:
 - Use Price (state) objects
 - State design pattern

Replace Conditional Logic



Replace Type Code with State

- Refactoring:
 - Replace price (type) code
 - Compile and test after each step
 - First, make sure uses of the price type code go through accessor methods ...

Replace Type Code with State

```
public class Movie {  
    ...  
    private int _priceCode;  
  
    public Movie( String title, int priceCode ) {  
        _title = title;  
        _priceCode = priceCode;  
    }  
    public int getPriceCode() {  
        return _priceCode;  
    }  
    public void setPriceCode( int arg ) {  
        _priceCode = arg;  
    }  
    ...  
}
```

Replace Type Code with State

```
public class Movie {  
    ...  
    private int _priceCode;  
  
    public Movie( String title, int priceCode ) {  
        _title = title;  
        setPriceCode( priceCode );  
    }  
    public int getPriceCode() {  
        return _priceCode;  
    }  
    public void setPriceCode( int arg ) {  
        _priceCode = arg;  
    }  
    ...  
}
```


Replace Type Code with State

- Refactoring:
 - Add new state classes ...

Replace Type Code with State

```
abstract class Price {
    public abstract int getPriceCode();
}

class RegularPrice extends Price {
    public int getPriceCode() {
        return Movie.REGULAR;
    }
}

class NewReleasePrice extends Price {
    public int getPriceCode() {
        return Movie.NEW_RELEASE;
    }
}

class ChildrensPrice extends Price {
    public int getPriceCode() {
        return Movie.CHILDRENS;
    }
}
```

Replace Type Code with State

- Refactoring:
 - Replace price type codes with instances of price state classes ...

Replace Type Code with State

```
public class Movie {  
    ...  
    private int _priceCode;  
    ...  
    public int getPriceCode() {  
        return _priceCode;  
    }  
    public void setPriceCode( int arg ) {  
        _priceCode = arg;  
    }  
    ...  
}
```

Replace Type Code with State

```
public class Movie {  
    ...  
    private Price _price;  
    ...  
    public int getPriceCode() {  
        return _price.getPriceCode();  
    }  
    public void setPriceCode( int arg ) {  
        switch (arg) {  
            case REGULAR:  
                _price = new RegularPrice();  
                break;  
            case NEW_RELEASE:  
                _price = new NewReleasePrice();  
                break;  
            case CHILDRENS:  
                _price = new ChildrensPrice();  
                break;  
            default:  
                throw new IllegalArgumentException(  
                    "Incorrect price code" );  
        }  
    }  
    ...  
}
```

Move Method

- Refactoring:
 - Move `getCharge()` to `Price` class

Move Method (Before)

```
public class Movie {  
    ...  
    public double getCharge( int daysRented ) {  
        double result = 0;  
        switch (getPriceCode()) {  
            case Movie.REGULAR:  
                result += 2;  
                if (daysRented > 2)  
                    result += (daysRented - 2) * 1.5;  
                break;  
            case Movie.NEW_RELEASE:  
                result += daysRented * 3;  
                break;  
            case Movie.CHILDRENS:  
                result += 1.5;  
                if (daysRented > 3)  
                    result += (daysRented - 3) * 1.5;  
                break;  
        }  
        return result;  
    }  
    ...  
}
```

Move Method (After)

```
public class Price {  
    ...  
    public double getCharge( int daysRented ) {  
        double result = 0;  
        switch (getPriceCode()) {  
            case Movie.REGULAR:  
                result += 2;  
                if (daysRented > 2)  
                    result += (daysRented - 2) * 1.5;  
                break;  
            case Movie.NEW_RELEASE:  
                result += daysRented * 3;  
                break;  
            case Movie.CHILDRENS:  
                result += 1.5;  
                if (daysRented > 3)  
                    result += (daysRented - 3) * 1.5;  
                break;  
        }  
        return result;  
    }  
    ...  
}
```


Move Method (After)

```
public class Movie {  
    ...  
    public double getCharge( int daysRented ) {  
        return _price.getCharge( daysRented );  
    }  
    ...  
}
```

Replace Conditional with Polymorphism

- Refactoring:
 - Replace switch statement in `getCharge()`
 - Define abstract method
 - For each case, add overriding method

Replace Conditional with Polymorphism (Before)

```
class Price {  
    ...  
    public double getCharge( int daysRented ) {  
        double result = 0;  
        switch (getPriceCode()) {  
            case Movie.REGULAR:  
                result += 2;  
                if (daysRented > 2)  
                    result += (daysRented - 2) * 1.5;  
                break;  
            case Movie.NEW_RELEASE:  
                result += daysRented * 3;  
                break;  
            case Movie.CHILDRENS:  
                result += 1.5;  
                if (daysRented > 3)  
                    result += (daysRented - 3) * 1.5;  
                break;  
        }  
        return result;  
    }  
    ...  
}
```

Replace Conditional with Polymorphism (After)

```
class RegularPrice {
    public double getCharge( int daysRented ) {
        double result = 2;
        if (daysRented > 2)
            result += (daysRented - 2) * 1.5;
        return result;
    }
}
class NewReleasePrice {
    public double getCharge( int daysRented ) {
        return daysRented * 3;
    }
}
class ChildrensPrice {
    public double getCharge( int daysRented ) {
        double result = 1.5;
        if (daysRented > 3)
            result += (daysRented - 3) * 1.5;
        return result;
    }
}
```

Replace Conditional with Polymorphism (After)

```
class Price {  
    ...  
    public abstract double getCharge( int daysRented );  
    ...  
}
```

Move Method

- Refactoring:
 - Move `getFrequentRenterPoints()` to `Price` class

Move Method (Before)

```
public class Movie {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        if ((getPriceCode() == Movie.NEW_RELEASE) &&  
            daysRented > 1)  
            return 2;  
        else  
            return 1;  
    }  
    ...  
}
```

Move Method (After)

```
class Price {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        if ((getPriceCode() == Movie.NEW_RELEASE) &&  
            daysRented > 1)  
            return 2;  
        else  
            return 1;  
    }  
    ...  
}
```


Move Method (After)

```
public class Movie {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        return _price.getFrequentRenterPoints( daysRented );  
    }  
    ...  
}
```

Replace Conditional with Polymorphism

- Refactoring:
 - Replace if statement in `getFrequentRenterPoints()`

Replace Conditional with Polymorphism (Before)

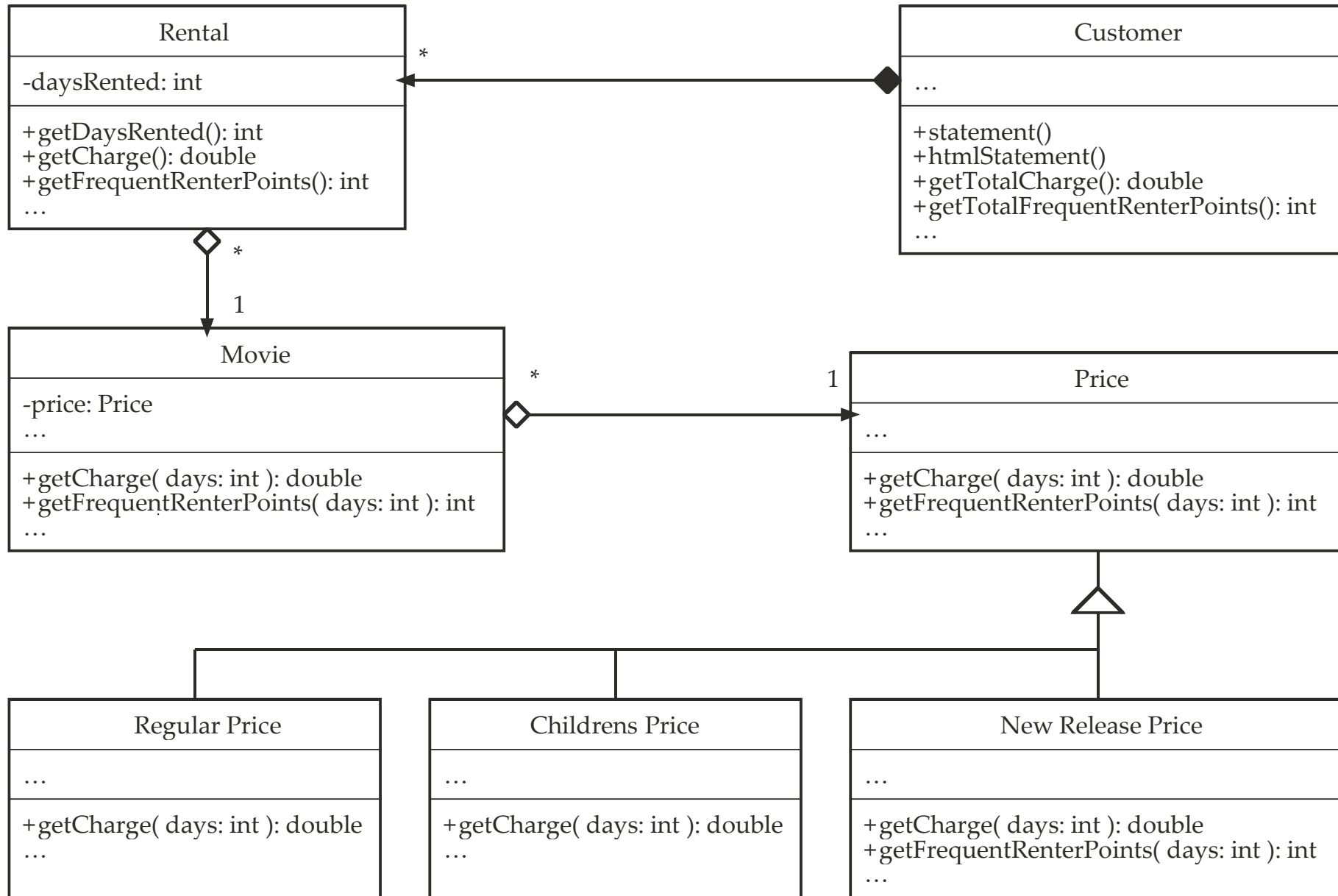
```
class Price {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        if ((getPriceCode() == Movie.NEW_RELEASE) &&  
            daysRented > 1)  
            return 2;  
        else  
            return 1;  
    }  
    ...  
}
```

Replace Conditional with Polymorphism (After)

```
class Price {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        return 1;  
    }  
    ...  
}  
  
class NewReleasePrice {  
    ...  
    public int getFrequentRenterPoints( int daysRented ) {  
        return (daysRented > 1) ? 2 : 1;  
    }  
    ...  
}
```

Refactoring

- Result:
 - Easier to change price behavior
 - Change movie classifications
 - Change rules for charging and frequent renter points
 - Rest of application does not know about this use of the State design pattern



Refactoring Principles

Refactoring

- Basic principles:
 - Catalog of refactorings
 - Do not change outward behavior
 - Reduce risk of change
 - One thing at a time
 - Test each step
 - Iterate

Refactoring

- Outcomes:
 - Encode design intent within class structure
 - Reorganizing code
 - Sharing logic
 - Express conditional logic

Refactoring

- Potential limitations:
 - Too much indirection
 - Performance impact
 - Changing published interfaces
 - Are significant design changes possible?

Refactoring

- When not to refactor:
 - When you should rewrite
 - When you are close to a deadline

Refactoring

- An analogy:
 - Unfinished refactoring is like going into debt
 - Debt is fine if you can meet the interest payments (extra maintenance costs)
 - If there is too much debt, you will be overwhelmed
 - Ward Cunningham

Kinds of Refactorings

- Creating methods:
 - Intended to help reduce the size of methods and improve the readability of the code
 - Extract Method, Inline Method, Replace Temp with Query

Kinds of Refactorings

- Moving features between objects:
 - Sometimes, responsibility is placed in the wrong class, or a class ends up with too many responsibilities
 - Move Method, Move Field, Extract Class

Kinds of Refactorings

- Organizing data:
 - Sometimes, objects can be used instead of simple data items
 - Replace Data Value with Object, Replace Array with Object

Kinds of Refactorings

- Simplifying conditional expressions:
 - Conditional expressions and logic can be difficult to understand
 - Replace Conditional with Polymorphism

Kinds of Refactorings

- Making method calls simpler:
 - Complicated programming interfaces can be difficult to use
 - Rename Method, Add Parameter

Kinds of Refactorings

- Dealing with generalization:
 - Getting methods and subclasses to the right place
 - Pull Up Method, Push Down Method, Extract Subclass, Extract Superclass